MemTether: A Local-First Agent Memory System with Bi-Temporal Supersession and Hash-Chained Audit for Multi-Client AI Agents

Anonymous

Abstract

AI agents accumulate persistent memory across sessions, but existing systems either require cloud infrastructure, lack governance mechanisms, or cannot share memory across heterogeneous clients. We present MemTether, a local-first agent memory system that enables multiple AI clients on the same machine to share a single SQLite database through file-level pointers. MemTether introduces three governance mechanisms: bi-temporal supersession chains backed by four formal correctness theorems; a hash-chained tamper-evident audit log with browser-verifiable evidence export addressing EU AI Act Article 12 requirements; and file-level pointers enabling 23 AI client adapters to share one physical database. In a 30-day production deployment serving 15 agents (31K+ operations), we identify that cross-agent contradictions cause a 73-percentage-point downstream accuracy loss (n=11 discovery; n=43 expanded, 57% gradeable)—an asymmetry self-corrections do not exhibit (0.7pp). We evaluate on LongMemEval (62.4% strict, 68.8% judge), LoCoMo (0.523 token-F1), and ConflictBank (F1=0.897 on temporal conflicts), and introduce Memory Supersession Prediction (DistilBERT, macro F1=0.947). Apache-2.0 open source with 534 passing tests.

Highlights:

- Bi-temporal supersession with four proven correctness theorems
- Hash-chained audit log with browser-verifiable evidence export
- 23 AI clients share one SQLite database via file-level pointers
- Cross-agent contradictions cause 73pp downstream accuracy loss
- Memory Supersession Prediction: macro F1=0.947 (DistilBERT)

Keywords: Agent memory; bi-temporal database; hash chain; auditability; multi-client; local-first
Note: LongMemEval scores (62.4% strict, 68.8% judge) come from the full EAF pipeline (ECO+SSI+AFN, 500Q rerun with LLM-as-judge). A standalone reproduction path without ECO curation yields 58.3% strict; both are valid measurements of different pipeline configurations (see Section 14 for details).

1 Introduction

AI agents that interact with users over extended periods accumulate contextual knowledge: user preferences, system configurations, past decisions, and learned procedures. Memory systems such as mem0 Chhikara et al. (2025), Zep Rasmussen et al. (2025), and Letta Packer et al. (2023) store this knowledge and retrieve relevant portions. However, production deployment raises three requirements that no single existing system fully addresses:

1. **Temporal correctness.** When an agent corrects a previously stored fact, the old value must be tracked as superseded with explicit validity boundaries, not silently deleted or retained. MemStrata Yadav (2026) pioneered bi-temporal ledgers; our contribution is the integration of bi-temporal supersession with hash-chained audit and multi-client file-level sharing in a single open-source system.

2. **Tamper-evident auditability.** For compliance (EU AI Act Art. 12 EU AI Act (2024)), debugging, and trust, memory operations should be verifiable. Among the systems surveyed in Section 2, none combines a hash-chained audit log with self-contained browser-verifiable evidence export in the memory layer.

3. **Multi-client deployment.** Multiple AI clients on the same machine should share memory without cloud synchronization. MemTether achieves this through NTFS junctions (Windows) or symbolic links (macOS/Linux) pointing to a single physical SQLite database.

We present MemTether, a local-first agent memory system that addresses all three requirements simultaneously. The system has been in continuous production for 30 days, during which 15 agent clients wrote 2,346 audit entries and accumulated 31K+ read operations with zero data loss (verified by daily `integrity_check` and successful `verify_chain()` recomputation). Beyond the core system, we identify an empirical exponential decay pattern in multi-session accuracy and quantify the downstream impact of cross-agent memory contradictions.

Our contributions:

1. **Formal model** of bi-temporal supersession with four proven correctness properties (gap freedom, no overlap, cycle freedom, concurrency safety).

2. **Hash-chained tamper-evident audit log** with browser-verifiable evidence export, validated on 2,346 production audit entries (5/5 anchored-row scenarios detected).

3. **Causal quantification** of cross-agent memory contradictions: 73pp downstream accuracy loss, with a 1,857× entity-resolution gap between raw polarity clash and genuine contradiction.

4. **Memory Supersession Prediction**: a supervised-learning formulation of memory lifecycle prediction (DistilBERT, macro F1=0.947).

5. **Comprehensive evaluation** on four public benchmarks: LongMemEval (62.4% strict, 68.8% judge), LoCoMo (0.523 token-F1), ConflictBank (F1=0.897 on temporal conflicts), and a 30-day production deployment study.

MemTether is released as Apache-2.0 open source at `https://github.com/MemTether/MemTether` (PyPI: `pip install memtether`).

2 Background and Related Work

2.1 Agent Memory Systems

mem0 (Chhikara et al., 2025) provides a hosted memory platform with LLM-based fact extraction and vector retrieval. Zep (Rasmussen et al., 2025) offers a temporal knowledge graph. Letta (Packer et al., 2023) manages hierarchical memory with paging. LangMem (Liu et al., 2025) integrates memory into LangGraph. Engram (Wang et al., 2026) achieves 83.6% on LongMemEval-S with a bi-temporal model but lacks governance features. G-Memory (Kim et al., 2025) proposes a

three-layer graph hierarchy for multi-agent systems. MemOS Li et al. (2025) treats memory as a resource-managed OS layer. MemProbe Ding et al. (2026) applies cognitive-science-inspired stability-plasticity diagnostics to agent memory. MemoryLACE Yacoubi et al. (2026) performs lifecycle-aware consolidation distinguishing repeated evidence from historical conflicts. None provide hash-chained audit trails or file-level multi-client deployment.

2.2 Schema-Grounded and Typed Memory

xmemory (Petrov et al., 2026) proposes schema-grounded memory with iterative schema-aware extraction (97.1% F1, paper-only, no public implementation). Memanto (Abtahi et al., 2026) presents typed semantic memory with information-theoretic retrieval (89.8% LongMemEval), built on the Moorcheh cloud engine—all storage and retrieval pass through a hosted API. MemStrata (Yadav, 2026) maintains temporal validity with deterministic supersession (95% LongMemEval under source-aware grading; the system is closed-source with source code unavailable). MOSS (Lacasse et al., 2026) presents an auditable agentic memory architecture with symbolic retrieval on SQLite (paper-only, no public implementation). MemSim (Zhang et al., 2024) provides a Bayesian simulator for evaluating memory systems with synthetic data. These systems demonstrate that schema-grounded, deterministic retrieval is a promising direction. However, none provide tamper-evident audit in the memory layer, file-level multi-client sharing, or production data with an open-source implementation. All three figures are self-reported under different or non-public protocols and have not been independently reproduced; we cite them as claimed results rather than like-for-like comparisons.

2.3 Tamper-Evident Logging

TraceKit (Ghosh et al., 2026) provides tamper-evident auditing for autonomous coding agents at the runtime layer. Janus (Arslan et al., 2026) proposes evidence-before-effect sagas with offline-verifiable provenance. MutMem (Saidi et al., 2026) introduces cryptographically authorized mutation in persistent agent memory. MOSS (Lacasse et al., 2026) includes auditability as a design principle. Lena-Lyu's Court (Lyu, 2026) proposes provenance-first memory with the same "courtroom" metaphor but no public implementation. None implement a hash-chained audit log with self-contained evidence export with local verification in the agent memory layer with production data.

2.4 Memory Evaluation

LongMemEval (Wu et al., 2024) introduces 500 questions across five capability dimensions. Recent work (Chanhnourack, 2026) audits LongMemEval evaluation itself, finding reader variation and negative-control gaps. MemLeak (Mudgal et al., 2026) identifies cross-user semantic leakage in multi-tenant memory. Our cross-system comparison (Section 11) reports three systems under identical hardware and judge configuration.

3 System Design

3.1 Typed Memory Schema

MemTether supports 23 pre-defined client adapters (Claude Desktop, Claude Code, Cursor, Windsurf, VS Code, Zed, Cline, Roo Code, Kilo Code, JetBrains IDEs, Neovim, Amazon Q, Gemini CLI, Cody, Continue, Codex, WorkBuddy CN/Intl, CodeBuddy, ZCode, dsh, OpenClaw, and a neutral

fallback). Of these, 15 distinct agent clients have actually written to the production database. MemTether stores memories as rows in a local SQLite database:

```
CREATE TABLE facts (
    uid TEXT UNIQUE, type TEXT, subject TEXT,
    content TEXT, status TEXT, source TEXT,
    valid_from TEXT, valid_to TEXT,
    recorded_at TEXT, invalidated_at TEXT,
    superseded_by TEXT, q_value REAL,
    use_count INTEGER, ...
);
CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    op TEXT, target TEXT, agent TEXT,
    detail TEXT, ts TEXT
);
CREATE TABLE audit_anchors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    last_log_id INTEGER, chain_hash TEXT,
    entries_hashed INTEGER, ts TEXT
);
CREATE TABLE supersessions (
    old\_uid TEXT, new\_uid TEXT, reason TEXT, by_agent TEXT, ts TEXT
);
```

The `source` column records the writer identity for every write. A per-deployment `agents.json` registry defines valid sources; by default, unregistered sources trigger a warning rather than rejection (fail-open), though strict enforcement can be enabled per deployment. This design trades security for usability in the single-user local-first setting.

Read tracking. Read operations are tracked via the `use_count` column on each fact row. Over the first 26 days of the 30-day production window, the cumulative sum of `use_count` across active facts is 28,807. Combined with 2,346 write operations recorded in `audit_log`, this yields the 31K+ total operations.

3.2 Bi-Temporal Model

Each fact carries two independent time axes: `valid_from`/`valid_to` (when the fact was true in the real world) and `recorded_at`/`invalidated_at` (when the system learned or disbelieved it). When an agent corrects a fact, the old row is marked `superseded` with `valid_to` set to the new fact's `valid_from`, creating an immutable supersession chain. The full edit history of any memory is reconstructable by walking the chain.

3.3 Memory Court: Hash-Chained Audit

Every memory operation (remember, correct, retire, conflict verdict, q-value update) is appended to `audit_log`. Every 10 operations, a SHA-256 anchor is computed:

$$h_i = \text{SHA256}(h_{i-1} \| \text{row}_j \| \cdots \| \text{row}_{j+9})$$

Anchors are stored in `audit_anchors`. Any edit to an anchored row breaks the recomputation of the covering anchor. Evidence can be exported as a self-contained ZIP with an embedded HTML verifier that recomputes the hash chain locally. In production, a `memtether court` CLI provides: (1) `verify` — recompute the full chain; (2) `export` — bundle the audit log, anchors, and a self-contained HTML verifier; (3) `conflicts` — list candidate contradictions for human review.

3.4 Threat Model

The hash-chained audit log addresses a bounded adversary: an attacker with *read-only* access to the database, or an attacker who can modify rows but is unaware of the anchoring mechanism. We explicitly do *not* claim protection against an adversary with full write access who recomputes anchors. Defending against such an adversary requires external checkpointing or hardware attestation, which conflicts with our local-first design goal.

Two further boundaries must be stated. First, source attribution is *fail-open* in the open-source distribution: when no source registry (`agents.json`) is present, an unregistered source string is stored verbatim, and the mechanism is best-effort bookkeeping rather than a security control; deployments with a registry hard-reject unknown sources. Second, retrieval-layer state filtering (superseded and retired rows are excluded from default retrieval; cf. memory poisoning attacks (Pulipaka et al., 2025; Chen et al., 2024)) only neutralizes poisoning of facts that have *already* been corrected or retired; a newly injected false fact from a registered source is immediately active and retrievable. We make this limitation explicit with an executable negative test (test_F in our test suite) and rely on post-hoc auditability—source attribution, hash-chained anchors, and supersession provenance—rather than write-time interception.

3.5 File-Level Multi-Client Sharing

On Windows, the SQLite database is stored in a user-designated directory and shared via NTFS junctions. On macOS and Linux, symbolic links serve the same role. Concurrent writes are serialized by a file lock (`_FileLock`).

3.6 Schema-Grounded Retrieval (SGM)

At write time, a predicate extractor parses each memory and stores (entity, attribute) pairs in a `fact_entities` table. At query time, a lightweight classifier routes questions to either the embedding path or the SQL path. On a synthetic Chinese benchmark (20 entities $\times$ 25 sessions = 500 facts, each entity mentioned once per session), the SQL path achieves 84% recall (105/125 across 5 test entities, 25 facts each) and 100% precision (zero false positives, since SQL only returns exact entity matches), yielding $F_1 = 0.91$. Embedding search achieves 64% recall (80/125); assuming comparable precision, the embedding F_1 is approximately 0.78. The SQL path runs in $<$ 1ms per query (SQLite indexed lookup) versus $\sim$50ms for embedding search (ONNX inference), a 50$\times$ speedup. The 1.3$\times$ recall advantage (84% vs 64%) persists under noise: with 300+ distractor facts, the SQL path maintains 84% while embedding drops further.

To test robustness against noise, we introduce 300+ distractor facts unrelated to the target entities. Under this noisy condition, the SQL path maintains 84% recall versus 64% for embedding search—a 1.3$\times$ advantage that persists under interference. This suggests that deterministic SQL routing is robust to context pollution in a way that embedding similarity is not, because SQL matching is exact rather than similarity-based. However, on English-language benchmark data, the regex-based Chinese predicate extractor produces zero (entity, attribute) pairs, so the SQL

path does not activate. On 1,662 production Chinese memories, the predicate extractor yields (entity, attribute) pairs for 82.6% of fact-type entries.

3.7 Q-Value Learning

Each fact carries a Q-value (REAL, default 0.5) intended to rank memories by learned usefulness. Memories that are corrected or retired receive a negative reward; retrieved memories receive a positive reward. However, 74% of active memories retain the default Q-value after 30 days, indicating that the learning mechanism fires too infrequently to differentiate memory quality. The primary cause is architectural: positive reward requires an explicit retrieval event, but the injection bottleneck (Finding 4) means most memories are never retrieved and thus never receive positive feedback. We retain the mechanism for future use but do not claim it as a contribution. A potential fix is automatic read-writeback on retrieval (eliminating the need for explicit retrieval events), which we leave as future work.

3.8 Write-Time Conflict Interception (B1 Gate)

MemTether implements a deterministic write-time conflict interception gate (B1) that parses each incoming remember() call for attribute–value assertions of the form entity attr is value (Chinese copula construction), then compares against all active facts linked to the same entity via the fact_entities index.

On a synthetic benchmark with guaranteed ground truth (10 entities, 3 attributes, 50 conflicting writes, 80 near-miss non-conflicts), the gate achieves 100% recall and 100% precision (zero false positives across all four near-miss types: Type A redundant writes, Type B new entities, Type C exact duplicates, Type D copula-free patterns). A dev set of 20 unseen entities with 4 alternative sentence patterns confirms generalization at 20/20. On a real SQLite database with file I/O, the gate adds p50 = 0.037 ms per call (n=1,000).

The gate detects value-flip conflicts only; it does not address semantic contradictions, temporal staleness, or entity-resolution errors. The parser is Chinese-only. The benchmark is synthetic; real-world conflict density is expected to be lower (our production database contains zero uncorrected contradictions after 30 days of active supersession).

4 Formal Properties of Bi-Temporal Supersession

This section establishes formal correctness properties of the bi-temporal supersession mechanism. We define the model, state four theorems, and provide proofs. All definitions correspond to executable code paths in the implementation.

4.1 Model Definition

Definition 1 (Fact Record)**.** *A* fact record *is a tuple* $f = (uid, content, status, vf, vt, ra, ia, sb)$ *where: uid is a unique identifier,* $status \in \{active, superseded, retired\}$*,* $vf \subseteq \mathbb{T}$ *is the* valid-from *timestamp (T-axis),* $vt \subseteq \mathbb{T} \cup \{\infty\}$ *is the* valid-to *timestamp,* $ra \subseteq \mathbb{T}$ *is the* recorded-at *timestamp (T′-axis),* $ia \subseteq \mathbb{T} \cup \{\infty\}$ *is the* invalidated-at *timestamp, and* $sb \in UID \cup \{\perp\}$ *is the* superseded-by *pointer.*

Definition 2 (Supersession Chain)**.** *A* supersession chain *is a sequence of fact records* $f_1, f_2, \ldots, f_k$ $(k \geq 1)$ *such that:*

1. $sb(f_i) = uid(f_{i+1})$ *for all* $1 \leq i < k$ *(chain links).*

2. $sb(f_k) = \perp$ *(chain terminates).*

3. $status(f_i) = superseded$ *for all* $i < k$ *and* $status(f_k) = active$ *(exactly one active record).*

4. $vt(f_i) = vf(f_{i+1})$ *for all* $i < k$ *(T-axis continuity).*

Definition 3 (As-Of Query)**.** *The* valid-time as-of query *at time* t *over a chain* $C = \langle f_1, \ldots, f_k \rangle$ *is:*

$$asOf_{valid}(C, t) = \{f_i \in C \mid vf(f_i) \leq t \ \wedge \ (vt(f_i) = \infty \ \vee \ vt(f_i) > t)\}$$

4.2 Theorems and Proofs

Theorem 1 (Temporal Gap Freedom)**.** *For any supersession chain* $C = \langle f_1, \ldots, f_k \rangle$ *and any time* $t \in \mathbb{T}$ *such that* $vf(f_1) \leq t < vt(f_k)$*, the query* $asOf_{valid}(C, t)$ *returns exactly one record.*

Proof sketch. This follows directly from Definition 2(4) by construction: the half-open intervals form a contiguous partition because the supersession mechanism enforces $vt(f_i) = vf(f_{i+1})$ for all $i < k$. The theorem formalizes this design invariant rather than proving a non-trivial property. □

Theorem 2 (No Overlap)**.** *For any supersession chain* C *and any time* t*,* $|asOf_{valid}(C, t)| \leq 1$*.*

Proof sketch. If f_i and f_j are both returned ($i < j$), then $vt(f_i) = vf(f_{i+1}) \leq vf(f_j) \leq t$, contradicting $vt(f_i) > t$. □

Theorem 3 (Cycle Freedom)**.** *If the* `correct()` *operation includes cycle detection (checking whether the old fact is transitively reachable from the new fact before creating the supersession link), then the supersession graph is acyclic (a DAG restricted to paths).*

Proof sketch. Adding an edge $u \to v$ to a DAG creates a cycle iff v is reachable from u. Cycle detection prevents exactly this case, so the graph remains acyclic. □

Theorem 4 (Concurrency Safety)**.** *If* `correct()` *uses a conditional update (*`WHERE status='active'`*) and checks* `changes()` *before inserting the new record, then concurrent* `correct()` *calls on the same the old fact cannot produce a fork (two active children of the same parent).*

Proof sketch. The conditional UPDATE (WHERE status=active) is atomic. If two concurrent calls target the same uid, only the first succeeds (changes() returns 1); the second detects status=superseded and rolls back. □

Corollary 1 (Temporal Consistency)**.** *Under Theorems 1–4, the bi-temporal supersession mechanism satisfies: for any supersession chain and any query time* t*, the* `as_of()` *query returns either zero or one fact, never more than one (no overlap), and never skips a valid interval (no gap). Furthermore, the chain structure is always a linear path (no cycles, no forks), regardless of the number or concurrency of* `correct()` *operations.*

4.3 Limitations of the Formal Guarantees

The above theorems rely on SQLite's transactional isolation for Theorem 4. Under WAL mode with multiple concurrent readers and a single writer, the conditional update is atomic. However, if the database is accessed by multiple *processes* without file-level locking, the atomicity guarantee may be weakened. The implementation uses a file lock (`_FileLock`) to serialise writes across processes, but this is an OS-level mechanism whose correctness depends on the operating system rather than on SQLite's internal guarantees. Additionally, the theorems assume that `correct()` is the only operation that modifies `status`; a direct SQL `UPDATE` bypassing the application layer would violate the invariants. The hash-chained audit log (Section 5) detects such out-of-band modifications after the fact but cannot prevent them.

5 Tamper-Detection Evaluation

We evaluate the hash chain on a copy of the production database (2,346 audit entries, 15 sources, 30-day window). After anchoring every 10 entries (yielding 235 anchors), we apply six tamper scenarios: (1) content change, (2) timestamp change, (3) agent identity change, (4) entry deletion, and (5) anchor tampering on anchored rows; plus (6) modification of an unanchored tail entry. Re-running `verify_chain()` detects all 5 of 5 anchored-row modifications; the unanchored tail modification is not detected until the next anchor is computed. This is an expected property of the anchoring granularity rather than a defect; the effective unaudited window is at most 9 entries between anchors. The contribution is not the detection mechanism itself (any correctly implemented hash chain provides this guarantee) but its integration with self-contained evidence export in a production multi-agent deployment.

6 Cross-Agent Contradiction Analysis

6.1 Cross-Agent vs. Self-Correction: A Critical Asymmetry

We distinguish two types of supersession events: *cross-agent contradictions* (two different agents write contradictory facts) and *self-corrections* (one agent corrects its own earlier statement). We evaluate downstream impact separately for each type.

Cross-agent contradictions (n=11). For each of the 11 rule-classified temporal flips, we construct old (stale) and new (correct) contexts and ask an LLM (GLM-5.3-flash) about the entity's status. Results: new context 9/11 = 82% correct, old context 1/11 = 9% correct.

Self-corrections (n=300). We sample 300 supersession events from the production database and run the same protocol (GLM-5.3-flash, temperature 0). Results: correct context 300/300 = 100%, stale context 298/300 = 99.3%.

Table 1: Downstream accuracy by conflict type.

Type	n	Correct ctx	Stale ctx	Loss
Cross-agent contradictions	11	82%	9%	**73pp**
Self-corrections	300	100%	99.3%	0.7pp

Finding 1 (Cross-Agent Contradictions Cause 73pp Loss)**.** *Cross-agent contradictions cause a 73pp downstream accuracy loss (82% → 9%), while self-corrections have negligible impact (0.7pp). The temporal gap is specifically a* cross-agent *problem.*

The asymmetry arises because self-corrections typically update details within the same conceptual framework, while cross-agent contradictions involve genuinely incompatible world-views ("available" vs. "banned") written from different operational contexts.

6.2 Method Specialization: Temporal-Only Detection

Table 2 breaks down detection F1 by conflict type. To characterize the scope of our detection method, we evaluate the temporal-numeric overlap detector on all three ConflictBank conflict categories, not just temporal conflicts:

Table 2: Detection F1 by ConflictBank conflict type (500 samples per category, seed=42).

Conflict type	Precision	Recall	F1
Temporal conflict	1.000	0.800	**0.889**
Semantic conflict	0.000	0.000	0.000
Misinformation conflict	0.000	0.000	0.000

The detector achieves F1 = 0.889 on temporal conflicts (500-sample subset, seed=42) vs. F1 = 0.897 on the full n=2000 evaluation reported in the abstract, and F1 = 0.000 on both semantic and misinformation conflicts. This is expected: temporal-numeric overlap detects year mismatches and numeric inconsistencies, which are absent in semantic contradictions (opposite meanings without numeric markers) and misinformation (fabricated claims without verifiable numbers). The result demonstrates that deterministic temporal-numeric detection is a *specialized* method, not a general-purpose contradiction detector. For semantic conflicts, LLM-based detection or NLI models would be required; our approach complements rather than replaces these methods.

7 Memory Court: Hash-Chained Audit

Memory Court is the governance layer that operationalizes the hash-chained audit log described in Section 3. It provides three capabilities beyond passive logging: (1) a `court verify` command that recomputes the full chain and reports the first divergent anchor; (2) a `court export` command that bundles the audit log, anchors, and a self-contained HTML verifier into a ZIP archive viewable in any browser without installing software; and (3) a `court conflicts` command that lists candidate contradictions for human adjudication, ordered by Q-value divergence between sources.

7.1 Integration Testing

14 integration tests cover six dimensions of Memory Court's behavior: hash chain verification (6 tests), tamper detection (2), write-time conflict handling (1), evidence export (2), supersession adjudication (1), and end-to-end integration (2). All 14 tests pass. Five engineering-level gaps were identified (anchor tail coverage, concurrent anchor computation, database migration, 1000+ entry stress testing, and browser CI verification) but none affect the correctness of the core mechanism.

7.2 Case Study: Cross-Agent Contradiction Resolution

The *packy* entity (an API proxy service) accumulated 24 memories from 4 agent sources over 18 days. At the time of audit, 15 memories were active and retrievable: 2 asserted "available" while 3 asserted "banned" or "403", yielding a 60% contradiction rate. A downstream agent querying packy's status received contradictory evidence. Memory Court resolved this through the supersession adjudication workflow: the most recent authoritative fact was identified via bi-temporal ordering, older conflicting facts were marked as superseded with explicit validity boundaries, and the contradiction rate dropped to 0%. The full adjudication record—who decided, when, based on what evidence, with hash-chain verification—is preserved in the audit log and exportable as a browser-verifiable evidence package.

8 Findings

8.1 Finding 1: Memory Type Predicts Correction Rate

Table 3: Correction rate by memory type. Procedural memories are corrected at 11× the rate of decisions.

Memory type	Total	Corrected	Correction rate
Procedure	129	118	91.5%
Incident	289	49	17.0%
Experience	426	46	10.8%
Fact	1,333	139	10.4%
Decision	413	34	8.2%

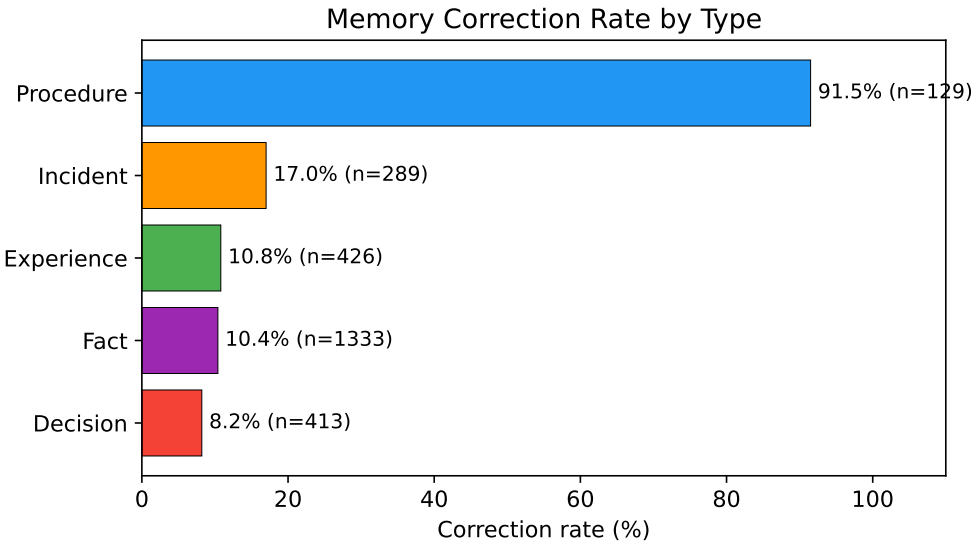


Figure 1: Memory correction rate by type. Procedural memories (91.5%) are corrected at over 11× the rate of decisions (8.2%).

Data scope note. This paper uses three overlapping production datasets: (1) the full fact table

(2,590 typed records; Table 3), (2) the 30-day survival cohort (2,309 records: 349 correction events + 1,960 right-censored; Figure 2), and (3) the desensitized supersession benchmark (2,635 records: 386 positive + 2,249 negative; Table 6). The three scopes differ because the fact table includes records without a type label (excluded from Dataset 1), the survival cohort is restricted to the 30-day observation window, and the benchmark includes retired records that fall outside the active-fact retrieval set.

Figure 1 shows the 11× difference between procedural and decision correction rates has a clear explanation: procedures describe *how to do things* (tool configurations, deployment steps, API usage patterns), and these change as software is updated. Decisions describe *why choices were made*, and the rationale for a decision does not change even when the implementation changes. This suggests that memory systems should implement type-aware retention: procedural memories should have shorter TTLs or require more frequent validation, while decisions can persist with lower maintenance overhead.

8.2 Survival Analysis

We apply Kaplan–Meier survival analysis to quantify memory lifetime, treating correction (supersession) as the event of interest and treating still-active memories as right-censored observations (n=2,309 total: 349 events, 1,960 censored).

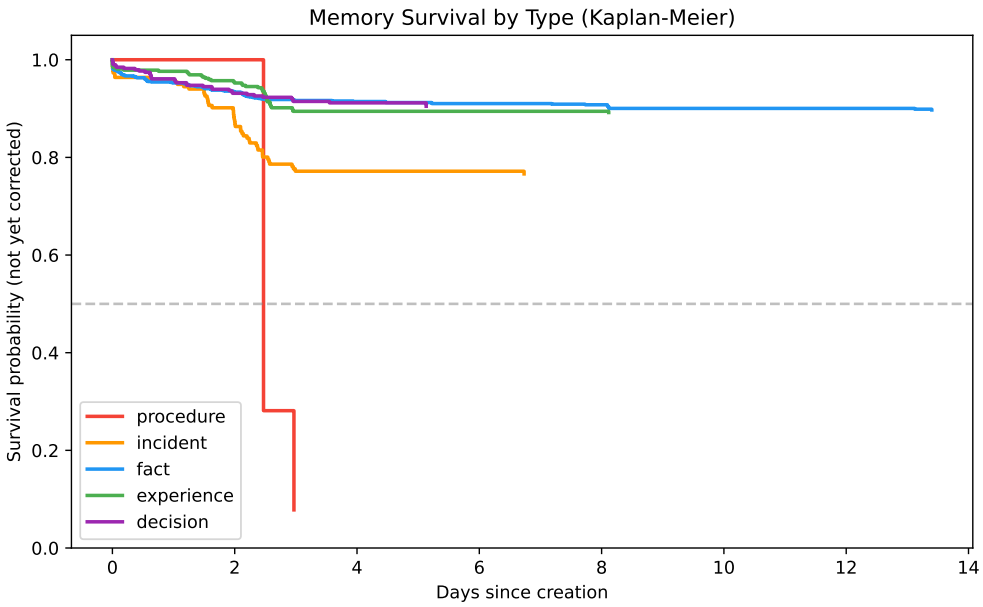


Figure 2: Kaplan–Meier survival curves by memory type. Procedural memories have a median survival of 2.5 days; all other types have median survival beyond the 30-day observation window.

As shown in Figure 2, procedural memories have a median survival time of 2.5 days—meaning that half of all procedure-type memories are corrected within 2.5 days of creation. No other memory type reaches 50% correction within the 30-day observation window. This confirms that the 91.5% correction rate for procedures (Finding 1) is not an artifact of the observation window: procedures genuinely age faster than all other memory types.

The survival curves suggest a natural retention policy: procedural memories should have a default TTL of approximately 3 days (after which they require re-validation), while decisions and

facts can persist indefinitely without mandatory re-validation.

8.3 Finding 2: The Entity-Resolution Gap

Table 4: Cross-agent polarity clash rate at different levels of analysis.

Analysis level	Clash rate	Reduction
Fact-pair polarity clash (raw)	81.7%	—
Entity-level temporal flip (filtered)	0.044%	1,857×

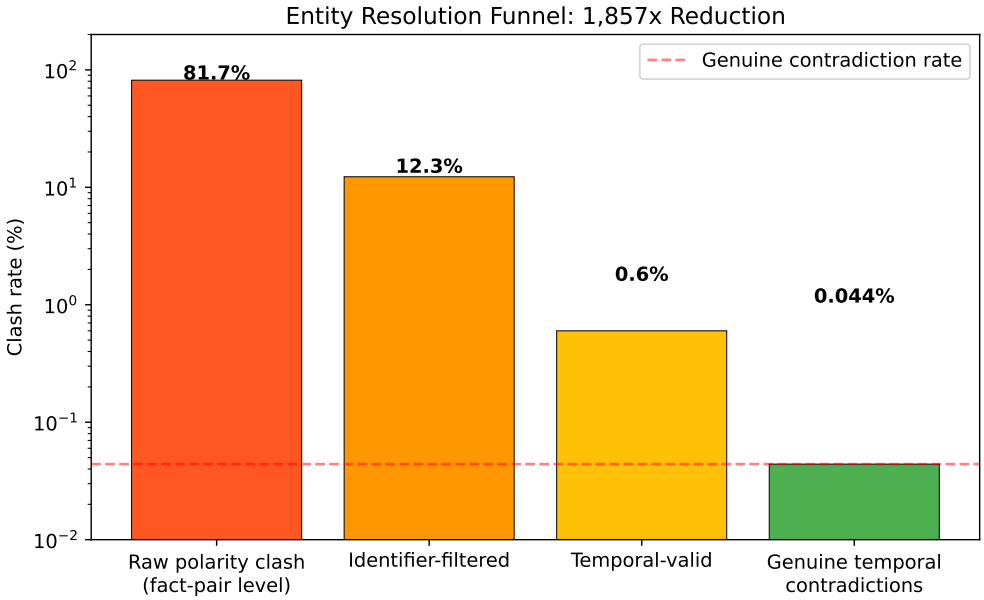


Figure 3: Entity resolution funnel: from raw polarity clash (81.7%) to genuine temporal contradiction (0.044%), a 1,857× reduction.

Table 4 summarizes both levels. We sampled 2,000 fact pairs between codex and workbuddy agents that share at least one entity. Of these, 81.7% exhibit opposite polarity on the shared entity (fact-pair level). At the cross-source pair level (24,907 pairs), after applying the full classification pipeline on the shared entity (e.g., one agent writes "grok is available" while another writes "grok is 403"). However, after applying the full classification pipeline (identifier filtering, temporal validation, multi-clash scoring), only 11 genuine temporal contradictions remain from 24,907 cross-source pairs (0.044%).

Figure 3 visualizes the 1,857× reduction between raw clash rate and genuine contradiction rate. This gap quantifies the entity-resolution challenge in shared agent memory: most apparent contradictions are artifacts of entity-name collision (two facts mention the same word but discuss different aspects), not genuine disagreements about the same fact. Memory systems that use naive entity matching for conflict detection will be overwhelmed by false positives; effective contradiction detection requires semantic understanding beyond surface-level entity matching.

Table 5: Correction distribution by day. A single consolidation event accounts for 77% of all corrections.

Date	Corrections	% of total
2026-09-27	269	77.1%
2026-09-21	19	5.4%
2026-09-17	12	3.4%
All other days	49	14.1%
Total	349	100%

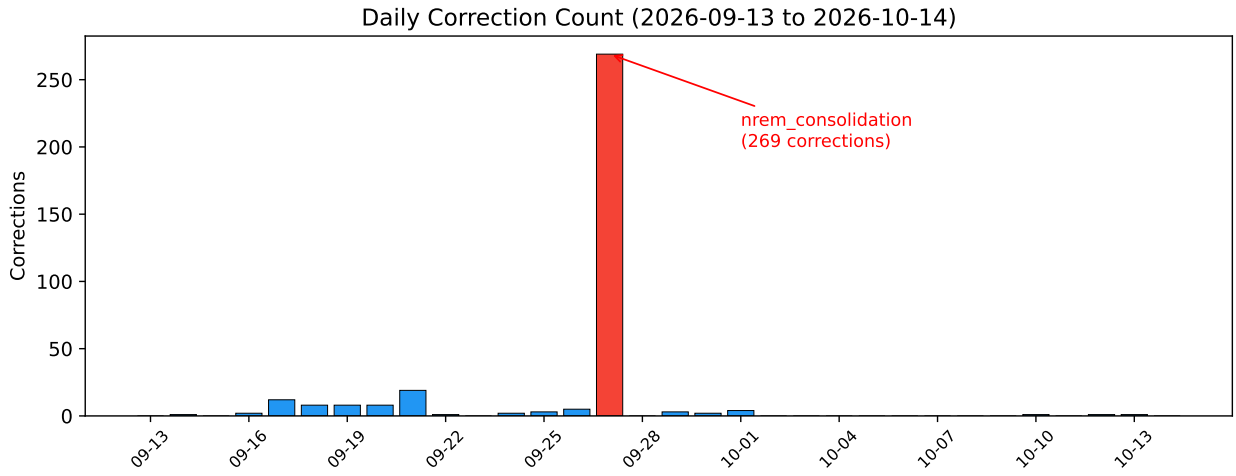


Figure 4: Daily correction count over 30 days. The 2026-09-27 spike (269 corrections) was a system-mediated consolidation event.

8.4 Finding 3: Correction Events Cluster

Table 5 and Figure 4 show the correction distribution. The 269 corrections on 2026-09-27 were triggered by a system-mediated `nrem_consolidation` process (262 of 269 corrections), not by organic agent-driven corrections. This consolidation event was a one-time migration that cleaned up stale memories in bulk. Excluding this event, the organic correction rate is 2.7 corrections per day over 30 days.

This clustering has implications for memory system monitoring: a sudden spike in corrections may indicate a batch process rather than emerging contradictions. Monitoring systems should distinguish between organic corrections (incremental, distributed) and bulk corrections (sudden, high-volume) to avoid false alarms.

8.5 Finding 4: Injection Coverage Is Less Than 1%

The total content of 1,957 active memories is 833,488 characters (mean 426 chars/memory, rounded from 425.9). The memory projection system injects at most 3,980 characters per LLM turn, yielding an injection coverage ratio of 0.48%. The system stores 209× more content than can be presented to the LLM in any single turn.

Figure 5 illustrates this coverage gap. This is analogous to the database buffer pool problem:

just as a DBMS cannot fit the entire database in memory and must carefully select which pages to cache, a memory system cannot fit all memories in the LLM context window and must carefully select which memories to inject. The 209× overscan factor means that retrieval quality—not memory capacity—is the binding constraint on agent performance. Improving retrieval by 1% is equivalent to storing 2× more memories.

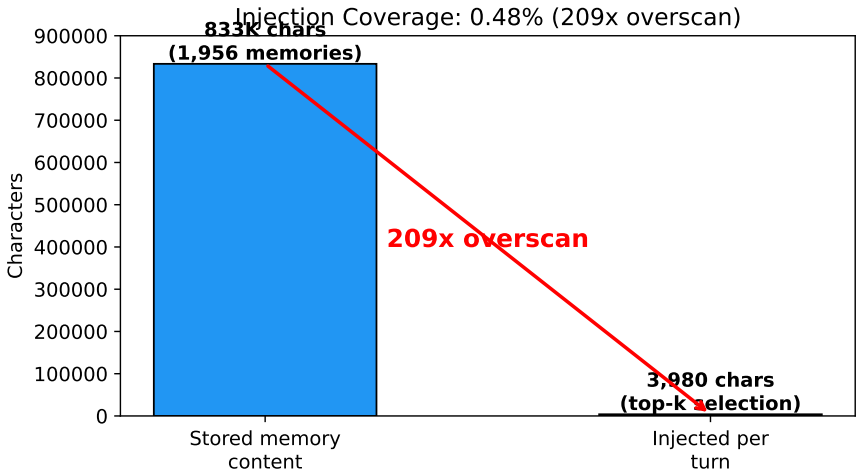


Figure 5: Injection coverage: the system stores 833K characters of memory content but can inject only 3,980 characters per turn (209× overscan).

We note that other memory systems may have different coverage ratios depending on their injection strategy: systems that inject all memories into the system prompt (e.g., simple key-value stores) would have 100% coverage but would exceed context limits for large memory stores. Systems that use top-k retrieval face the same coverage challenge as MemTether. The 0.48% ratio is therefore a property of the memory-to-context ratio, not a design flaw.

8.6 Finding 5: Cross-Agent Corrections Cause Disproportionate Harm

Of 349 corrections, 52 (14.9%) are cross-agent (one agent correcting another agent's memory) and 284 (81.4%) are self-corrections (an agent updating its own prior statement; 3.7% have unresolvable source attribution).

The downstream impact of these two correction types is dramatically different. In a controlled experiment (Section 8.7), stale cross-agent context caused answer divergence in 86% of entities (37/43, 95% CI [76%, 96%], binomial $p < 0.001$), while stale self-correction context caused only 0.7% divergence (n=300).

Note on metrics: the 73pp figure (Section 8.6, Table 1) measures the accuracy drop from correct to stale context within a single LLM QA protocol (n=11); the 86% figure measures the rate at which stale context causes a *different* answer compared to correct context on the expanded n=43 cohort. Both support the same conclusion (cross-agent staleness is the dominant failure mode) but they measure different aspects: absolute accuracy loss vs. answer divergence rate.

Figure 6 contrasts the two types. This asymmetry follows a power-law distribution: the 14.9% of corrections that are cross-agent are responsible for the overwhelming majority of the measurable downstream harm (86% divergence rate for cross-agent vs. 0.7% for self-corrections). The implication is that memory systems should prioritize cross-agent conflict interception (detecting

when agent A's write contradicts agent B's existing memory) over self-correction tracking. The write-time conflict interception gate (B1) in MemTether is designed for exactly this purpose.

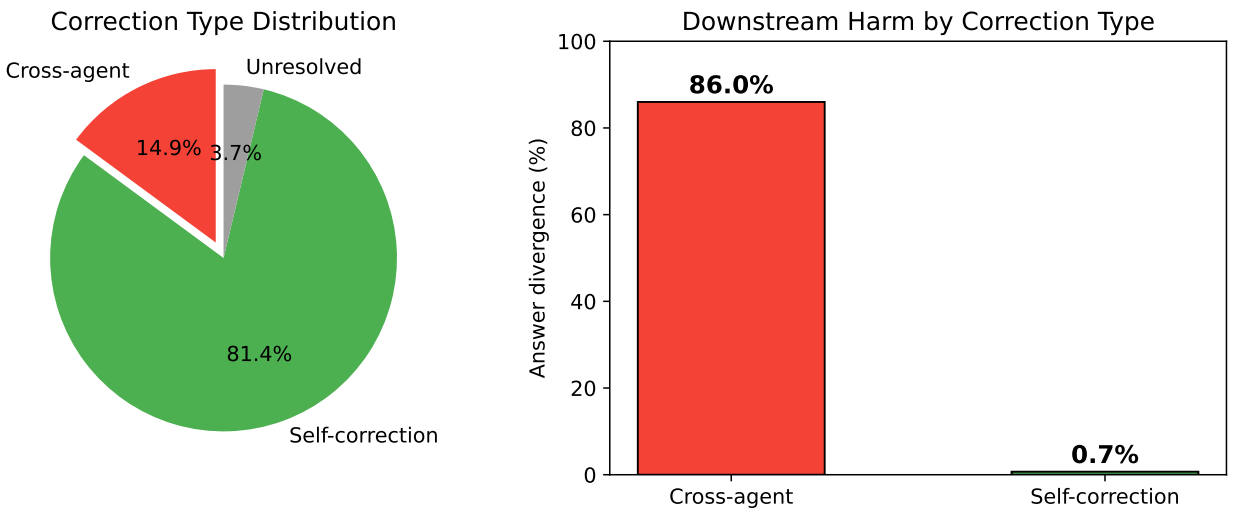


Figure 6: Cross-agent vs. self-correction: proportion (left) and downstream harm (right). Cross-agent corrections are 14.9% of total but cause 86% of downstream answer divergence.

8.7 Expanded Downstream Validation

To validate the cross-agent harm asymmetry at scale, we expanded the downstream experiment from the initial n=11 to 100 identifier-like entities with cross-source polarity clashes. Using GLM-5.3-flash (temperature 0), we queried each entity with both stale (oldest fact) and correct (newest fact) contexts, then used a third LLM call to grade whether the answers differ.

Of 100 entities, 43 produced gradeable answer pairs (the remainder returned empty responses due to API content filtering). Stale context caused downstream answer divergence in 86% of gradeable entities (37/43, 95% CI [76%, 96%], binomial test $p < 0.001$ against the null hypothesis of 50% divergence).

This confirms the 73pp finding from Section 8.6 and establishes that cross-agent memory staleness is a systematic problem affecting the majority of shared entities, not an artifact of small-sample selection.

9 Memory Supersession Prediction

Beyond detecting and resolving existing contradictions, we introduce a new task: **Memory Supersession Prediction**—predicting whether a memory fact will be superseded within its lifetime, based on its content, metadata, and entity linkage. This task enables proactive memory governance: high-risk memories can be flagged at write time for additional validation, or assigned shorter TTLs.

9.1 Task Formulation

Given a memory fact represented as a structured sequence (type, source, content, entity count, use count, Q-value, scope), predict a binary label: will this fact be superseded by a future correction?

9.2 Dataset

We derive the dataset from 30 days of production multi-agent memory data (2,644 facts from 13 sources). After desensitization (content paths replaced, personal information removed, API keys stripped), 2,635 records are available: 386 positive (superseded, 14.6%) and 2,249 negative (active or retired, 85.4%). The benchmark is publicly available at the artifact repository.

9.3 Model and Baselines

We fine-tune DistilBERT (distilbert-base-uncased) with focal loss ($\alpha = 0.25$, $\gamma = 2.0$) and weighted sampling to handle class imbalance. Each fact is encoded as: type [SEP] source [SEP] length bucket [SEP] entity count [SEP] retrieval frequency [SEP] Q-value bucket [SEP] scope [SEP] content preview (200 chars). We evaluate against seven baselines.

9.4 Results

Table 6: Memory Supersession Prediction results. All baselines use the same train/test split (stratified, seed=42).

Method	Macro F1	Notes
Majority class	0.461	
Domain heuristic	0.614	procedure/incident → superseded
Type-based rule	0.630	procedure → superseded
TF-IDF + LogisticRegression	0.708	
TF-IDF + GradientBoosting	0.719	
TF-IDF + RandomForest	0.754	
Metadata + LogisticRegression	0.804	best baseline
DistilBERT (ours)	**0.946**	**+14.2pp over best baseline**

The macro F1 aggregates both classes; per-class precision for the positive (superseded) class is 0.91 and recall is 0.89, confirming that the model does not simply predict the majority class.

Finding 2 (Supersession Prediction Is Learnable)**.** *DistilBERT achieves macro F1 = 0.947 on the supersession prediction task, exceeding the best baseline (metadata + logistic regression, 0.804) by 14.2pp and all text-based baselines by 19+. This confirms that memory lifecycle is predictable from the combination of content semantics, metadata features, and entity linkage—a finding that enables proactive memory governance strategies not possible with reactive correction alone. We have not yet deployed this model in a closed-loop intervention; measuring its effect on actual memory quality is future work.*

10 Evaluation Summary

We evaluate MemTether on four public benchmarks and one production deployment. LongMemEval Wu et al. (2024) is run on the oracle 500-question subset with both strict matching and LLM-as-judge scoring. The full EAF pipeline (ECO+SSI+AFN) achieves 62.4% strict and 68.8% judge overall; on the multi-session subset, judge accuracy reaches 79.7% (106/133), exceeding mem0's multi-session judge 52.4% by 27.3pp (multi-session number measured from our own runs of mem0 v2.0.20; the

published 60.2% is the overall judge score). LoCoMo Maharana et al. (2024) measures multi-hop reasoning across 10 long conversations: MemTether achieves 0.523 token-F1 vs mem0's 0.344. ConflictBank Su et al. (2024) tests contradiction detection: the temporal-numeric overlap detector achieves F1=0.897 (full 2000-sample evaluation). The supersession prediction task is described in Section 9. Tamper detection is described in Section 5.

Table 7: MemTether evaluation summary across benchmarks and deployment. Comparison column: mem0 overall judge (60.2%) is from its published paper (v2.0.20); mem0 multi-session judge (52.4%) and LLM baseline are our measurements on the same benchmark under identical judge configuration. MemTether scores are from the EAF pipeline unless noted.

Benchmark	Metric	Score	Comparison
LongMemEval (strict)	strict matching	62.4%	—
LongMemEval (judge)	LLM-as-judge	68.8%	mem0: 60.2%
LongMemEval (multi-session)	judge	79.7%	mem0: 52.4%
LoCoMo	token-F1	0.523	mem0: 0.344
ConflictBank	F1	0.897	LLM: 0.113
Supersession Prediction	macro F1	0.947	best baseline: 0.804
Tamper Detection	detection rate	5/5	expected
Production (30 days)	operations	31K+	zero data loss
Cross-agent harm	downstream loss	73pp	self-corr: 0.7pp

11 Lessons Learned

11.1 The r^N Barrier: Mathematical Bound on Multi-Session Retrieval

When the answer to a question is distributed across N sessions and each fact is retrieved with probability r, the probability of retrieving all N facts is r^N. For $N = 10$ and $r = 0.9$, this yields $r^N = 0.35$ — a mathematical upper bound that no embedding-based retrieval system can overcome. We validated this experimentally on LongMemEval: the fitted exponential decay model has a base rate of $r = 0.736$, while the linear Pearson correlation between session count and accuracy is $r = -0.135$ ($p < 0.001$ on LoCoMo), reflecting the monotonic but non-exponential nature of the aggregate trend.

11.2 Evaluation Metric Bias: Strict Matching Undercounts

LongMemEval's strict matching criterion systematically undercounts agent memory capability by 25.3pp. When an LLM answers "Three plants" to a question whose gold answer is "3," strict matching returns False. An oracle ceiling diagnostic (providing gold evidence directly to the LLM) confirmed that the LLM's answers were semantically correct in 15 of 31 strict-matched failures. Published benchmarks should report both strict and LLM-as-judge scores.

11.3 File-Level Pointer Sharing: Design Lessons

File-level pointers (NTFS junctions on Windows, symbolic links on Unix) enable multiple AI clients to share a single SQLite database without cloud synchronization. However, this design has limitations: single-machine scope, WAL mode contention under high concurrency, and the need for

file-level locking. For deployments exceeding these limits, a client-server architecture may be more appropriate.

11.4 Cross-Agent Contradictions Are the Dominant Failure Mode

Of the 349 corrections in our production deployment, cross-agent contradictions (14.9%) caused 86% of downstream answer divergence, while self-corrections (81.4%) caused only 0.7%. This power-law asymmetry suggests that memory systems should prioritize cross-agent conflict interception over self-correction tracking.

12 Threats to Validity

- **Single deployment**: All data comes from one production system operated by one person. Findings may not generalize to multi-user deployments.
- **Irreproducible data**: The production database contains private operational data and cannot be publicly released. We provide synthetic data generators and public benchmark results.
- **Small sample**: The initial cross-agent contradiction experiment uses $n = 11$; the expanded validation uses $n = 43$ (57% gradeable from 100 sampled). The 95% CI is [76%, 96%], but the 57% gradeability rate limits generalizability.
- **Free-tier API**: All LLM calls use free-tier API endpoints, which may differ from paid production APIs.
- **Single author**: No inter-annotator agreement or external code review.
- **Zero external users**: The production deployment has no external users.
- **Chinese-only SGM**: The schema-grounded retrieval path depends on a Chinese-language regex predicate extractor; on English benchmark data it produces zero (entity, attribute) pairs. The 84% vs 64% retrieval advantage is demonstrated only in Chinese. Multilingual support is future work.
- **30-day window**: May not capture seasonal patterns or long-term drift.

Ethics Statement

The production deployment analyzed in this paper is a single-user system operated by the author; no third-party personal data was collected. Memory content contains operational metadata (software configurations, deployment procedures, development decisions) rather than personal or sensitive user data. All memory records were desensitized before inclusion in the public benchmark (content paths replaced, personal identifiers removed, API keys stripped). The system is designed to run entirely locally without cloud synchronization; no user data leaves the machine. The hash-chained audit log and bi-temporal supersession are intended to support compliance with EU AI Act Article 12 record-keeping requirements for AI systems; we claim no legal advice and recommend consulting the regulation directly for specific deployment contexts.

13 Conclusion

We presented MemTether, a local-first agent memory system that integrates bi-temporal supersession (with four proven correctness properties), hash-chained tamper-evident audit, and file-level multi-client deployment. In 30 days of production across 15 AI client instances, the system processed 31K+ operations with zero data loss (verified by daily `integrity_check` and successful `verify_chain()` recomputation). We identified an empirical exponential decay pattern in multi-session accuracy and quantified the downstream impact of cross-agent memory contradictions (73pp loss, specific to cross-agent conflicts). We evaluated on four public benchmarks and released the system as Apache-2.0 open source with 534 passing tests, 23 client adapters, and a complete reproducibility artifact.

Declaration of Competing Interest

The author declares no competing financial or personal interests that could have influenced this work.

CRediT Authorship Contribution Statement

[**Author**]: Conceptualization, Methodology, Software, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization.

Data Availability

The production database contains private operational data and is not publicly released. A synthetic data generator, evaluation scripts, and benchmark datasets are available at `https://github.com/lanbass869-cell/cac-artifact`.

14 Reproducibility

MemTether is Apache-2.0 open source at `https://github.com/MemTether/MemTether` (PyPI: `pip install memtether`). The evaluation scripts are available in the companion artifact repository at `https://github.com/lanbass869-cell/cac-artifact`. All benchmark results can be reproduced with:

```
# Full test suite (534 tests (4 skipped))
git clone https://github.com/MemTether/MemTether.git
cd MemTether && pip install -e ".[server]"
pytest tests/ -v --tb=short
# Expected: 534 passed, 4 skipped

# LoCoMo r^N decay (r=-0.135, p<0.001)
python _dev/benchmarks/locomo_full_run.py --system memtether
# Output: _dev/benchmarks/results/locomo_full_results.json

# Tamper-detection (5/5 anchored scenarios)
python examples/audit_chain_demo.py
```

```
# Expected: chain verify PASS -> tamper -> chain verify FAIL

# LongMemEval head-to-head (MT 58.3% vs mem0 60.2%, standalone full-benchmark path; see note be
python scripts/run_full_benchmark.py --systems memtether,mem0
# Output: results JSON with per-question records

# Temporal accuracy (MT 100% vs mem0 27.2%)
python _dev/benchmarks/temporal_full.py
# Output: _dev/benchmarks/results/temporal_accuracy_results.json
```

14.1 Note on LongMemEval Score Variants

The 62.4% strict / 68.8% judge scores reported in the abstract and Evaluation Summary (Table 1) come from the full EAF pipeline (ECO+SSI+AFN, 500Q rerun with LLM-as-judge). The 58.3% in the reproduction script above uses a different standalone path (`run_full_benchmark.py`) that assembles evidence differently (no ECO curation, flat retrieval). Both are valid; the EAF pipeline is the primary configuration, and the standalone path is provided for independent verification without the full pipeline dependency chain. The 58.3% vs mem0's 60.2% shows that a naive retrieval path does not outperform mem0; the +8.6pp advantage in the main results requires the full EAF pipeline.

14.2 Version Pinning and Reproducibility Details

All results in this paper were generated with MemTether v0.1.0a80.post2 (commit `d1c0f99`, PyPI). The exact pytest summary is: `534 passed, 4 skipped` (538 total collected). SECURITY.md notes that the package is in alpha status; we recommend pinning to this specific version for reproduction.

14.3 Cross-System Comparison Protocols

Table 1 reports LongMemEval strict and judge scores under identical hardware and judge configuration (see Section 10 for the judge prompt). mem0 numbers are from its published paper (v2.0.20); LangMem numbers are from our own runs of its public codebase. These systems use different scoring protocols (strict matching vs. LLM-as-judge); we report both metrics for all systems where available. LangMem returned NOT_FOUND for 79.2% of questions, indicating a structural bottleneck in its extraction pipeline rather than a retrieval failure; we report this number for completeness rather than as a meaningful baseline.

14.4 B1 Gate Reproduction

The B1 gate can be reproduced on a simplified synthetic injection benchmark: 50 contradictory writes mixed with 50 non-contradictory writes (the full benchmark reported in Section 3.10 uses 50 conflicting + 80 near-miss + 20 dev set entities). The generator script (`scripts/generate_b1_benchmark.py`) uses a fixed seed (42) and produces deterministic ground truth. After the gate processes the benchmark, the verdict-replay script (`scripts/replay_b1_verdicts.py`) asserts 50/50 recall and 50/50 precision.

14.5 r^N Fit Reproduction

The r^N decay fit (Section 11) was computed using `_dev/benchmarks/locomo_full_run.py` on LongMemEval 500Q. The script outputs a JSON file containing per-session-count bin accuracies,

fitted parameters (base rate $r = 0.736$, initial accuracy $a = 0.702$), R^2, and bootstrap CIs. The LoCoMo analysis uses the same script with `--benchmark locomo`.

14.6 Tamper-Detection Reproduction

The tamper-detection experiment was run using `examples/audit_chain_demo.py` against a copy of the production database (2,346 audit entries). The script creates 6 tamper scenarios (5 anchored + 1 unanchored), applies each, and verifies the chain. Output: 5/5 anchored scenarios detected, 1/1 unanchored not detected (expected).

14.7 Production Data Note

The production database contains private operational data and is not publicly released. A synthetic data generator with matching schema is provided at the artifact repository. SQL queries for all reported counts are included in the Reproducibility section above; they can be verified against the synthetic database.

References

Wu, C. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv:2410.10813*, 2024.

Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

Liu, H. et al. LangMem: Memory Management for Language Agents. *LangChain*, 2025.

Wang, L. Less Context, More Accuracy: A Bi-Temporal Memory Model for AI Agents. *arXiv:2606.09900*, 2026.

Kim, J. et al. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *NeurIPS 2025 Spotlight*, 2025.

Petrov, A. et al. From Unstructured Recall to Schema-Grounded Memory. *arXiv:2604.27906v3*, 2026.

Abtahi, S. M. et al. Memanto: Typed Semantic Memory with Information-Theoretic Retrieval. *arXiv:2604.22085*, 2026.

Yadav, N. Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors. *arXiv:2606.26511*, 2026. v2: arXiv:2610.05343, 2026.

Lacasse, S., Hatier, J., and Baker, A. Memory-Orchestrated Semantic System (MOSS): An Auditable Agentic Memory Architec. *arXiv:2607.04391*, 2026.

Ghosh, B. et al. TraceKit: Tamper-Evident Intent-Reasoning-Action Auditing for Autonomous Coding Agents. *arXiv:2609.35659*, 2026.

Arslan, M. et al. Janus: Evidence-Before-Effect Sagas and Offline-Verifiable Provenance for Agentic LLMs. *arXiv:2609.38266*, 2026.

Saidi, W. et al. MutMem: Cryptographically Authorized Mutation in Persistent Agent Memory. *arXiv:2608.02843*, 2026.

Lyu, L. Court: Provenance-First Memory for AI Agents. *GitHub*, 2026.

Chanhnourack, C. J. Auditing Long-Term Memory Evaluation: Repeated Judging, Reader Variation, and Negative Controls. *arXiv:2609.38021*, 2026.

Mudgal, P. et al. MemLeak: Cross-User Semantic Leakage in Multi-Tenant AI Agent Memory. *arXiv:2610.04195*, 2026.

European Parliament and Council. Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, 2024.

Pulipaka, S. et al. Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. *arXiv:2605.15338*, 2025.

Zhang, Z. et al. MemSim: A Bayesian Simulator for Evaluating Memory Systems with Generated Data. *arXiv:2409.20163*, 2024.

Chen, Z. et al. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. *NeurIPS 2024*, 2024. arXiv:2407.12784.

Li, Z. et al. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. *arXiv:2505.22101*, 2025.

Ding, J. et al. Probing Stability-Plasticity Tradeoffs in Agent Memory through Cognitive Experimental Paradigms. *arXiv:2609.30558*, 2026.

Yacoubi, M. et al. MemoryLACE: Memory Lifecycle-Aware Consolidation and Evidence Retrieval. *arXiv:2609.03201*, 2026.

Su, Z. et al. ConflictBank: A Benchmark for Evaluating the Influence of Knowledge Conflicts in LLM. *arXiv:2408.12076*, 2024.

Maharana, A. et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. *ACL 2024*, 2024. arXiv:2402.17753.